

```

# Google Colab Bengali Character Hybrid Classifier
# Architecture:
# image -> LeNet stroke map -> ResNet input -> VGG input ->
EfficientNet input -> ViT input
# Final classifier fuses LeNet + ResNet + VGG + EfficientNet + ViT
logits.
#
# Expected zip layout:
# Dataset.zip
#   train/
#     class_1/
#       image001.bmp
#     class_2/
#       image002.bmp
#   test/
#     class_1/
#       image003.bmp
#     class_2/
#       image004.bmp
#
# In Colab:
# 1. Runtime > Change runtime type > GPU
# 2. Paste/run this file in one notebook cell.

import os
import random
import shutil
import time
import zipfile
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from google.colab import files
from PIL import Image
from sklearn.metrics import classification_report, confusion_matrix
from torch.cuda.amp import GradScaler, autocast
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms
from torchvision.datasets import ImageFolder

SEED = 42
ZIP_LIMIT_MB = 500
DATA_ROOT = Path("/content/bengali_hybrid_dataset")
OUTPUT_DIR = Path("/content/bengali_hybrid_results")

```

```

# If upload button does not appear, upload zip from Colab's left Files
panel
# and set ZIP_PATH = "/content/your_file.zip".
ZIP_PATH = ""
AUTO_USE_NEWEST_CONTENT_ZIP = True

MODEL_IMAGE_SIZE = 224
BATCH_SIZE = 8
EPOCHS = 8
LEARNING_RATE = 3e-4
WEIGHT_DECAY = 1e-4
NUM_WORKERS = 2
MAX_TRAIN_IMAGES = 40000
MAX_TEST_IMAGES = 10000
USE_PRETRAINED = True
FREEZE_BACKBONES = True

def seed_everything(seed: int) -> None:
    random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True

def choose_zip_file() -> Path:
    if ZIP_PATH:
        zip_path = Path(ZIP_PATH)
        if not zip_path.exists():
            raise FileNotFoundError(f"ZIP_PATH does not exist:
{zip_path}")
        return zip_path

    existing_zips = sorted(Path("/content").glob("*.zip"), key=lambda
p: p.stat().st_mtime, reverse=True)
    if AUTO_USE_NEWEST_CONTENT_ZIP and existing_zips:
        zip_path = existing_zips[0]
        print(f"Using existing zip from /content: {zip_path}")
        return zip_path

    print("Upload your train/test zip file now.")
    print("If the upload button does not appear, upload the zip from
Colab's left Files panel and rerun.")
    uploaded = files.upload()
    if not uploaded:
        raise RuntimeError("No zip uploaded. Set ZIP_PATH or upload a
zip to /content and rerun.")
    zip_name = next(iter(uploaded.keys()))
    return Path("/content") / zip_name

```

```

def upload_and_extract_zip() -> Path:
    zip_path = choose_zip_file()
    if zip_path.suffix.lower() != ".zip":
        raise ValueError("Please upload a .zip file.")

    size_mb = zip_path.stat().st_size / (1024 * 1024)
    if size_mb > ZIP_LIMIT_MB:
        raise ValueError(f"Zip is {size_mb:.1f} MB. Limit is
{ZIP_LIMIT_MB} MB.")

    if DATA_ROOT.exists():
        shutil.rmtree(DATA_ROOT)
    DATA_ROOT.mkdir(parents=True, exist_ok=True)

    with zipfile.ZipFile(zip_path) as archive:
        for member in archive.infolist():
            target = DATA_ROOT / member.filename
            if not
str(target.resolve()).startswith(str(DATA_ROOT.resolve())):
                raise ValueError(f"Unsafe path inside zip:
{member.filename}")
            archive.extractall(DATA_ROOT)

    print(f"Extracted to: {DATA_ROOT}")
    return DATA_ROOT

def find_split_dir(root: Path, split_name: str) -> Path:
    direct = root / split_name
    if direct.exists() and direct.is_dir():
        return direct

    candidates = [p for p in root.rglob("*") if p.is_dir() and
p.name.lower() == split_name]
    if not candidates:
        raise FileNotFoundError(f"Could not find a '{split_name}'
folder inside the zip.")
    return candidates[0]

def count_images(split_dir: Path) -> int:
    extensions = {".bmp", ".png", ".jpg", ".jpeg", ".tif", ".tiff"}
    return sum(1 for p in split_dir.rglob("*") if p.is_file() and
p.suffix.lower() in extensions)

class LimitedImageFolder(Dataset):
    def __init__(self, root: Path, transform, max_images:

```

```

Optional[int] = None):
    self.base = ImageFolder(str(root), transform=None)
    self.classes = self.base.classes
    self.class_to_idx = self.base.class_to_idx
    self.transform = transform

    samples = list(self.base.samples)
    if max_images and len(samples) > max_images:
        by_class: Dict[int, List[Tuple[str, int]]] = {}
        for sample in samples:
            by_class.setdefault(sample[1], []).append(sample)
        limited = []
        per_class = max(1, max_images // max(1, len(by_class)))
        rng = random.Random(SEED)
        for class_samples in by_class.values():
            rng.shuffle(class_samples)
            limited.extend(class_samples[:per_class])
        samples = limited[:max_images]

    self.samples = samples
    self.targets = [target for _, target in samples]

def __len__(self):
    return len(self.samples)

def __getitem__(self, index):
    path, target = self.samples[index]
    image = Image.open(path).convert("RGB")
    if self.transform:
        image = self.transform(image)
    return image, target

class LeNetStem(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=5, padding=2),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=5, padding=2),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
        )

```

```

def forward(self, x):
    return self.features(x)

class ViTFeatureExtractor(nn.Module):
    def __init__(self, vit_model: nn.Module):
        super().__init__()
        self.vit = vit_model
        self.hidden_dim = vit_model.hidden_dim

    def forward(self, x):
        n = x.shape[0]
        x = self.vit._process_input(x)
        batch_class_token = self.vit.class_token.expand(n, -1, -1)
        x = torch.cat([batch_class_token, x], dim=1)
        x = self.vit.encoder(x)
        return x[:, 0]

class HybridViTResNetVGGEfficientNetLeNet(nn.Module):
    def __init__(self, num_classes: int):
        super().__init__()
        self.lenet_stem = LeNetStem()
        self.lenet_pool = nn.AdaptiveAvgPool2d((4, 4))
        self.lenet_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.25),
            nn.Linear(256, num_classes),
        )
        self.lenet_to_image = nn.Sequential(
            nn.Conv2d(128, 32, kernel_size=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 3, kernel_size=1),
            nn.Tanh(),
        )

        resnet_weights = models.ResNet18_Weights.DEFAULT if
USE_PRETRAINED else None
        resnet = models.resnet18(weights=resnet_weights)
        self.resnet_features = nn.Sequential(*list(resnet.children())
[:-1])
        self.resnet_head = nn.Linear(resnet.fc.in_features,
num_classes)
        self.resnet_to_vgg_gate = nn.Sequential(
            nn.Linear(resnet.fc.in_features, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 6),
        )

```

```

        vgg_weights = models.VGG16_BN_Weights.DEFAULT if
USE_PRETRAINED else None
        vgg = models.vgg16_bn(weights=vgg_weights)
        self.vgg_features = vgg.features
        self.vgg_pool = vgg.avgpool
        self.vgg_reduce = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512 * 7 * 7, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.25),
        )
        self.vgg_head = nn.Linear(512, num_classes)
        self.vgg_to_effnet_gate = nn.Sequential(
            nn.Linear(512, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 6),
        )

        eff_weights = models.EfficientNet_B0_Weights.DEFAULT if
USE_PRETRAINED else None
        efficientnet = models.efficientnet_b0(weights=eff_weights)
        self.effnet_features = efficientnet.features
        self.effnet_pool = nn.AdaptiveAvgPool2d(1)
        eff_dim = efficientnet.classifier[1].in_features
        self.effnet_head = nn.Linear(eff_dim, num_classes)
        self.effnet_to_vit_gate = nn.Sequential(
            nn.Linear(eff_dim, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 6),
        )

        vit_weights = models.ViT_B_16_Weights.DEFAULT if
USE_PRETRAINED else None
        vit = models.vit_b_16(weights=vit_weights)
        self.vit_features = ViTFeatureExtractor(vit)
        self.vit_head = nn.Linear(vit.hidden_dim, num_classes)

        if FREEZE_BACKBONES:
            self.freeze_module(self.resnet_features)
            self.freeze_module(self.vgg_features)
            self.freeze_module(self.effnet_features)
            self.freeze_module(self.vit_features)

        self.fusion = nn.Sequential(
            nn.Linear(num_classes * 5, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.30),
            nn.Linear(512, num_classes),
        )

```

```

@staticmethod
def freeze_module(module: nn.Module) -> None:
    for param in module.parameters():
        param.requires_grad = False

@staticmethod
def apply_channel_gate(x: torch.Tensor, gate_values: torch.Tensor)
-> torch.Tensor:
    scale = torch.sigmoid(gate_values[:, :3]).view(-1, 3, 1, 1)
    bias = torch.tanh(gate_values[:, 3:]).view(-1, 3, 1, 1)
    return x * scale + bias

def forward(self, x):
    lenet_map = self.lenet_stem(x)
    lenet_logits = self.lenet_head(self.lenet_pool(lenet_map))

    lenet_image = nn.functional.interpolate(
        self.lenet_to_image(lenet_map),
        size=x.shape[-2:],
        mode="bilinear",
        align_corners=False,
    )
    resnet_input = x + 0.25 * lenet_image
    resnet_vec = self.resnet_features(resnet_input).flatten(1)
    resnet_logits = self.resnet_head(resnet_vec)

    vgg_input = self.apply_channel_gate(resnet_input,
self.resnet_to_vgg_gate(resnet_vec))
    vgg_map = self.vgg_features(vgg_input)
    vgg_vec = self.vgg_reduce(self.vgg_pool(vgg_map))
    vgg_logits = self.vgg_head(vgg_vec)

    effnet_input = self.apply_channel_gate(vgg_input,
self.vgg_to_effnet_gate(vgg_vec))
    effnet_map = self.effnet_features(effnet_input)
    effnet_vec = self.effnet_pool(effnet_map).flatten(1)
    effnet_logits = self.effnet_head(effnet_vec)

    vit_input = self.apply_channel_gate(effnet_input,
self.effnet_to_vit_gate(effnet_vec))
    vit_vec = self.vit_features(vit_input)
    vit_logits = self.vit_head(vit_vec)

    fused = torch.cat([lenet_logits, resnet_logits, vgg_logits,
effnet_logits, vit_logits], dim=1)
    return self.fusion(fused)

def get_transforms() -> Tuple[transforms.Compose, transforms.Compose]:

```

```

train_tfms = transforms.Compose(
    [
        transforms.Resize((MODEL_IMAGE_SIZE, MODEL_IMAGE_SIZE)),
        transforms.RandomAffine(degrees=8, translate=(0.05, 0.05),
scale=(0.95, 1.05)),
        transforms.ToTensor(),
        transforms.Normalize(mean=(0.485, 0.456, 0.406),
std=(0.229, 0.224, 0.225)),
    ]
)
test_tfms = transforms.Compose(
    [
        transforms.Resize((MODEL_IMAGE_SIZE, MODEL_IMAGE_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=(0.485, 0.456, 0.406),
std=(0.229, 0.224, 0.225)),
    ]
)
return train_tfms, test_tfms

```

```

def train_one_epoch(model, loader, criterion, optimizer, scaler,
device) -> Tuple[float, float]:
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        with autocast(enabled=device.type == "cuda"):
            logits = model(images)
            loss = criterion(logits, labels)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        total_loss += loss.item() * labels.size(0)
        correct += (logits.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

    return total_loss / max(1, total), correct / max(1, total)

```

```

@torch.no_grad()
def evaluate(model, loader, criterion, device) -> Tuple[float, float,

```



```

List[int], List[int]]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    y_true = []
    y_pred = []

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        logits = model(images)
        loss = criterion(logits, labels)
        preds = logits.argmax(dim=1)

        total_loss += loss.item() * labels.size(0)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        y_true.extend(labels.cpu().tolist())
        y_pred.extend(preds.cpu().tolist())

    return total_loss / max(1, total), correct / max(1, total),
    y_true, y_pred

def fit_model(num_classes: int, train_loader, test_loader, device):
    model =
HybridViTResNetVGGEfficientNetLeNet(num_classes).to(device)
    trainable_params = [p for p in model.parameters() if
p.requires_grad]
    print(f"Trainable parameters: {sum(p.numel() for p in
trainable_params):,}")

    optimizer = optim.AdamW(trainable_params, lr=LEARNING_RATE,
weight_decay=WEIGHT_DECAY)
    criterion = nn.CrossEntropyLoss()
    scaler = GradScaler(enabled=device.type == "cuda")

    best_acc = 0.0
    best_true = []
    best_pred = []
    started = time.perf_counter()

    for epoch in range(1, EPOCHS + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader,
criterion, optimizer, scaler, device)
        test_loss, test_acc, y_true, y_pred = evaluate(model,
test_loader, criterion, device)
        if test_acc >= best_acc:
            best_acc = test_acc

```

```

        best_true = y_true
        best_pred = y_pred
    print(
        f"epoch {epoch:02d}/{EPOCHS} "
        f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} "
        f"test_loss={test_loss:.4f} test_acc={test_acc:.4f}"
    )

    return {
        "model":
"Hybrid_LeNet_to_ResNet_to_VGG_to_EfficientNet_to_ViT",
        "accuracy": best_acc,
        "train_seconds": round(time.perf_counter() - started, 2),
        "status": "ok",
        "y_true": best_true,
        "y_pred": best_pred,
    }

def main() -> None:
    root = upload_and_extract_zip()
    seed_everything(SEED)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    print("Device:", device)
    if device.type != "cuda":
        print("Warning: GPU not detected. This hybrid is heavy and
will be slow on CPU.")

    train_dir = find_split_dir(root, "train")
    test_dir = find_split_dir(root, "test")

    print("\nDataset folders")
    print("Train:", train_dir)
    print("Test: ", test_dir)
    print("Train image count:", count_images(train_dir))
    print("Test image count: ", count_images(test_dir))

    train_tfms, test_tfms = get_transforms()
    train_dataset = LimitedImageFolder(train_dir, train_tfms,
MAX_TRAIN_IMAGES)
    test_dataset = LimitedImageFolder(test_dir, test_tfms,
MAX_TEST_IMAGES)

    if train_dataset.classes != test_dataset.classes:
        print("Warning: train/test class folders differ.")
        print("Train classes:", train_dataset.classes)
        print("Test classes: ", test_dataset.classes)

```

```

num_classes = len(train_dataset.classes)
print("\nClasses:", num_classes)
print("Train samples used:", len(train_dataset))
print("Test samples used: ", len(test_dataset))
print("\nHybrid path")
print("image -> LeNet -> ResNet -> VGG -> EfficientNet -> ViT ->
fused classifier")

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available(),
)
test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available(),
)

payload = fit_model(num_classes, train_loader, test_loader,
device)
leaderboard = pd.DataFrame([k: v for k, v in payload.items() if k
not in ["y_true", "y_pred"]])
leaderboard.insert(0, "rank", [1])

print("\nLeaderboard")
display(leaderboard)

leaderboard_path = OUTPUT_DIR /
"hybrid_vit_resnet_vgg_effnet_lenet_leaderboard.csv"
leaderboard.to_csv(leaderboard_path, index=False)
print("Saved leaderboard:", leaderboard_path)

class_names = train_dataset.classes
y_true_names = [class_names[i] for i in payload["y_true"]]
y_pred_names = [class_names[i] for i in payload["y_pred"]]

print(f"\nBest model: {payload['model']}")
print(f"Best accuracy: {payload['accuracy']:.4f}")
print("\nClassification report")
print(classification_report(y_true_names, y_pred_names,
zero_division=0))

cm = pd.DataFrame(
    confusion_matrix(y_true_names, y_pred_names,

```

```

labels=class_names),
    index=class_names,
    columns=class_names,
)
print("\nConfusion matrix")
display(cm)

cm_path = OUTPUT_DIR / "best_model_confusion_matrix.csv"
cm.to_csv(cm_path)
print("Saved confusion matrix:", cm_path)
files.download(str(leaderboard_path))

```

```

if __name__ == "__main__":
    main()

```

Using existing zip from /content: /content/Dataset (2).zip
 Extracted to: /content/bengali_hybrid_dataset
 Device: cuda

Dataset folders

Train: /content/bengali_hybrid_dataset/Dataset/train
 Test: /content/bengali_hybrid_dataset/Dataset/test
 Train image count: 12000
 Test image count: 3000

Classes: 50

Train samples used: 12000
 Test samples used: 3000

Hybrid path

image -> LeNet -> ResNet -> VGG -> EfficientNet -> ViT -> fused
 classifier

Downloading: "https://download.pytorch.org/models/vgg16_bn-6c64b313.pth" to /root/.cache/torch/hub/checkpoints/vgg16_bn-6c64b313.pth

100%|██████████| 528M/528M [00:24<00:00, 22.9MB/s]

Downloading:

"https://download.pytorch.org/models/efficientnet_b0_rwrightman-7f5810bc.pth" to
 /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwrightman-7f5810bc.pth

100%|██████████| 20.5M/20.5M [00:00<00:00, 122MB/s]

Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth

100%|██████████| 330M/330M [00:18<00:00, 18.6MB/s]

Trainable parameters: 14,120,769

```
/tmp/ipykernel_2772/183902700.py:402: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
  scaler = GradScaler(enabled=device.type == "cuda")
/tmp/ipykernel_2772/183902700.py:355: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with autocast(enabled=device.type == "cuda"):
```

```
epoch 01/8 train_loss=1.5415 train_acc=0.5483 test_loss=0.5377
test_acc=0.8230
epoch 02/8 train_loss=0.4604 train_acc=0.8498 test_loss=0.3514
test_acc=0.8867
epoch 03/8 train_loss=0.3273 train_acc=0.8927 test_loss=0.2560
test_acc=0.9143
epoch 04/8 train_loss=0.2603 train_acc=0.9129 test_loss=0.2451
test_acc=0.9163
epoch 05/8 train_loss=0.2169 train_acc=0.9260 test_loss=0.2095
test_acc=0.9237
epoch 06/8 train_loss=0.1961 train_acc=0.9335 test_loss=0.2202
test_acc=0.9337
epoch 07/8 train_loss=0.2102 train_acc=0.9314 test_loss=0.2294
test_acc=0.9283
epoch 08/8 train_loss=0.1775 train_acc=0.9407 test_loss=0.2174
test_acc=0.9277
```

Leaderboard

```
{"summary":{"\n  \"name\": \"    main()\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"rank\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"Hybrid_LeNet_to_ResNet_to_VGG_to_EfficientNet_to_ViT\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 0.9336666666666666,\n        \"max\": 0.9336666666666666,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          0.9336666666666666\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"train_seconds\",\n      \"properties\": {\n        \"dtype\": \"number\",
```

```

{"number": 3115.92, "std": null, "min": 3115.92, "max": 3115.92, "num_unique_values": 1, "samples": [3115.92], "semantic_type": "", "description": "", "column": "status", "properties": {"dtype": "string", "num_unique_values": 1, "samples": ["ok"], "semantic_type": "", "description": ""}}
{"type": "dataframe"}

```

Saved leaderboard:

/content/bengali_hybrid_results/hybrid_vit_resnet_vgg_effnet_lenet_leaderboard.csv

Best model: Hybrid_LeNet_to_ResNet_to_VGG_to_EfficientNet_to_ViT

Best accuracy: 0.9337

Classification report

	precision	recall	f1-score	support
172	1.00	0.60	0.75	60
173	0.75	1.00	0.86	60
174	1.00	0.98	0.99	60
175	0.95	1.00	0.98	60
176	0.94	0.75	0.83	60
177	0.82	0.98	0.89	60
178	0.92	0.97	0.94	60
179	0.98	0.93	0.96	60
180	0.83	1.00	0.91	60
181	1.00	0.90	0.95	60
182	0.95	0.90	0.92	60
183	0.92	0.98	0.95	60
184	0.98	0.87	0.92	60
185	0.83	0.97	0.89	60
186	0.87	0.90	0.89	60
187	1.00	0.87	0.93	60
188	0.94	0.97	0.95	60
189	0.89	0.98	0.94	60
190	0.98	0.97	0.97	60
191	0.93	0.95	0.94	60
192	1.00	0.97	0.98	60
193	0.92	0.98	0.95	60
194	0.93	0.93	0.93	60
195	1.00	0.95	0.97	60
196	1.00	0.93	0.97	60
197	0.94	0.77	0.84	60
198	0.92	0.95	0.93	60
199	1.00	0.92	0.96	60
200	0.97	0.98	0.98	60
201	0.97	0.97	0.97	60

202	0.82	0.75	0.78	60
203	0.96	0.92	0.94	60
204	0.92	0.97	0.94	60
205	1.00	0.95	0.97	60
206	0.82	0.93	0.88	60
207	0.86	0.98	0.91	60
208	0.94	0.82	0.88	60
209	0.97	0.97	0.97	60
210	0.95	0.88	0.91	60
211	0.93	0.95	0.94	60
212	0.92	0.97	0.94	60
213	0.84	0.93	0.88	60
214	0.94	1.00	0.97	60
215	1.00	1.00	1.00	60
216	1.00	0.98	0.99	60
217	0.98	1.00	0.99	60
218	0.98	0.97	0.97	60
219	0.98	0.97	0.97	60
220	1.00	1.00	1.00	60
221	0.97	0.93	0.95	60
accuracy			0.93	3000
macro avg	0.94	0.93	0.93	3000
weighted avg	0.94	0.93	0.93	3000
Confusion matrix				
{ "type" : "dataframe" }				
Saved confusion matrix:				
/content/bengali_hybrid_results/best_model_confusion_matrix.csv				
<IPython.core.display.Javascript object>				
<IPython.core.display.Javascript object>				